

Voya Activity SaaS User Guide

What each screen does, how screens are linked, and what the fields mean

This guide explains the vendor portal in plain language. It follows the real application screens and shows how a vendor moves from onboarding to activities, rate plans, availability, bookings, payouts, and performance.

Quick start

Item	Value
Frontend	http://localhost:3007
Backend API	http://localhost:4007/api
Swagger	http://localhost:4007/api/docs
Demo email	vendor@voya.demo
Demo password	Demo@123

The browser application is the front door. It calls the NestJS API, and the API reads and writes PostgreSQL through Prisma. You normally work in the browser; pgAdmin is useful when you want to inspect the stored rows.

Technical architecture

The demo has three runtime layers. The React and Vite frontend runs on port 3007 and owns screen state, form controls, validation messages, and navigation. The NestJS backend runs on port 4007 under the /api prefix and owns authentication, role checks, validation, business rules, and database access. PostgreSQL stores the tenant, vendor, catalogue, pricing, inventory, booking, payout, and audit data in the activity_saas database.

Layer	Technology	Responsibility	Demo evidence
Browser	React + Vite	Renders screens, collects input, stores the JWT, calls API endpoints	Menu changes screens without a page reload
API	NestJS + class-validator	Authenticates requests, validates DTOs, applies workflow rules	HTTP status and error messages in the UI
Data access	Prisma ORM	Maps typed service calls to PostgreSQL rows and relations	Saved values remain after refresh
Database	PostgreSQL 16	Persists vendor-owned business data	Rows visible in pgAdmin under activity_saas

End to end technical flow

Use this sequence when demonstrating the system. Each step produces a visible result and a corresponding API or database change.

Step	User action	Frontend request	Backend and database result
1. Authenticate	Enter demo email and password and select Sign in	POST /api/auth/login	NestJS verifies the password and returns a JWT. The browser stores it as voya_token.
2. Restore session	Refresh the browser	GET /api/auth/me	The JWT guard identifies the user. The app then loads GET

			/api/vendor/profile.
3. Complete vendor setup	Edit business fields and select Save Business Details	PATCH /api/vendor/profile	VendorProfile is updated for the logged-in tenant only.
4. Verify documents	Choose a PDF, JPG, JPEG, or PNG in a document control	POST /api/vendor/documents/{key}	The document filename, status, and upload time are stored in VendorProfile.documentStatus.
5. Create catalogue content	Select Listings, New Listing, enter fields, and save	POST /api/activities or PATCH /api/activities/{id}	An Activity row is created or updated. Highlights, terms, location, rating, and information arrays are normalized.
6. Add commercial rules	Open an activity and add a rate plan	POST /api/activities/{id}/rate-plans	RatePlan is created with TravellerRule and CancellationRule child rows.
7. Publish	Select Publish Listing on a draft activity	POST /api/activities/{id}/publish	The activity status changes to LIVE in the demo workflow.
8. Manage inventory	Choose a rate plan, edit available/capacity/price/closed, and save	POST /api/availability/bulk	AvailabilitySlot rows are updated with version checks to prevent stale overwrites.
9. Manage demand	Schedule a promotion or toggle one off	POST /api/promotions or PATCH /api/promotions/{id}	Promotion stores its time window, percentage, cap, and active flag.
10. Fulfil bookings	Confirm, cancel, or request a voucher	POST /api/bookings/{id}/confirm, /cancel, or GET /voucher	Booking status transitions are validated. Confirmed bookings can return a voucher code.

Logical business flow

The business logic moves from identity to supply, then from supply to demand and settlement. The dependency is intentional: a booking is meaningful only when it points to an activity and, where applicable, a rate plan; an availability slot is meaningful only when it belongs to a rate plan.

Business object	Owns or belongs to	Why it matters
Tenant	Users, VendorProfile, Activities, Bookings, Payouts	Defines the vendor boundary used for isolation
VendorProfile	Business details and documentStatus JSON	Controls readiness, verification, and payout identity
Activity	Media, RatePlans, Promotions, Bookings	Represents the sellable product or experience
RatePlan	TravellerRules, CancellationRules, AvailabilitySlots	Represents a specific price and operating contract
AvailabilitySlot	One rate plan, date, and start time	Controls capacity, remaining inventory, closure, and optional price
Booking	One tenant, activity, optional rate plan	Represents a reservation and its state transition
Payout	One tenant and settlement status	Represents money moving through scheduled, transit, or paid states

State transitions to demonstrate

Object	Starting state	Action	Result
Activity	DRAFT	Submit for Review	UNDER_REVIEW
Activity	DRAFT, INACTIVE, or	Publish Listing in the vendor	LIVE

	UNDER_REVIEW	demo	
Activity	LIVE	Edit and save	UNDER_REVIEW, so changes are reviewed
Booking	PENDING	Confirm	CONFIRMED
Booking	PENDING or CONFIRMED	Cancel	CANCELLED
Promotion	active = true	Toggle switch	active = false
AvailabilitySlot	open	Set Closed and save	closed = true; inventory remains auditable

Tenant isolation and security flow

Every protected request carries Authorization: Bearer <JWT>. The JWT identifies the user role and tenantId. NestJS guards reject missing or invalid tokens, the roles guard limits vendor routes to vendor users, and service queries add the current tenantId to their filters. This is why one vendor cannot list or modify another vendor's activities, bookings, availability, or payouts. ValidationPipe rejects unknown fields and invalid types before the service writes data.

Request lifecycle in detail

A normal save follows the same lifecycle on every screen. First, a React control changes local form state. When the user selects a save or action button, the API client serializes the state as JSON and adds the bearer token. NestJS receives the request under /api, authenticates it, validates the DTO, and calls the feature service. The service adds tenant ownership rules, performs business validation, writes through Prisma, and returns the updated row. React then replaces its local state with the response, which is why the screen immediately reflects the server result.

Stage	Component	What happens	Failure visible to the user
1. Input	React field or toggle	Value is held in component state and displayed immediately	The control remains editable
2. Submit	api.request	JSON body and Authorization header are sent	Network or authentication error
3. Authenticate	JwtAuthGuard	Token is decoded and the user is loaded	401 Unauthorized
4. Authorize	RolesGuard	Role metadata is checked against the route	403 Forbidden
5. Validate	ValidationPipe and DTO	Unknown fields, wrong types, invalid dates, and invalid ranges are rejected	400 Bad Request with a readable message
6. Apply rules	Feature service	Tenant ownership, status transition, and optimistic version rules run	409 Conflict or 404 Not Found
7. Persist	Prisma and PostgreSQL	Rows and related child records are inserted or updated	500 only for an unexpected server failure
8. Refresh state	React page	Returned data replaces stale local state	Success message or refreshed table

Detailed screen responsibilities

Screen	Reads from the API	Writes to the API	Logical purpose
Login	None before authentication	POST /auth/login	Creates the authenticated vendor session

Dashboard	GET /dashboard/summary	None	Shows a read-only operational overview
Onboarding	GET /vendor/profile	PATCH /vendor/profile; POST /vendor/documents/{key}	Maintains the vendor identity, readiness, payout display, and document status
Listings	GET /activities	POST /activities; PATCH /activities/{id}	Finds activities and opens the editor
Activity Editor	GET /activities/{id}; GET /activities/{id}/rate-plans	PATCH activity; media endpoints; rate-plan create; submit; publish	Maintains the product master and its commercial offers
Availability	GET /availability; GET /promotions	POST /availability/bulk; promotion create/toggle	Maintains saleable inventory and time-boxed discounts
Bookings	GET /bookings	Confirm, cancel, voucher	Runs the operational reservation queue
Payouts	GET /payouts	None	Displays settlement history
Performance	GET /dashboard/summary	None	Turns operational metrics into quality indicators

Example API messages

The following examples show the logical shape of the messages. The browser sends the request body; the server adds generated IDs, timestamps, tenant ownership, and version fields.

Create an activity

POST /api/activities with a body such as: { productName: "Sunrise Trek", type: "ACTIVITY", subType: "TICKET_ONLY", description: "Guided sunrise trek", shortDescription: "A guided morning experience", starRating: 4.8, cityName: "Munnar", stateName: "Kerala", countryName: "INDIA", address: "Meeting point", lat: 10.0889, lon: 77.0595, highlights: ["Guide included", "Sunrise viewpoint"], terms: ["Carry valid ID"], thingsToCarry: ["Walking shoes"], importantInfo: ["Weather dependent"] }. The response contains the new Activity id and status DRAFT.

Create a rate plan

POST /api/activities/{activityId}/rate-plans creates the commercial contract. The body contains ratePlanCode, name, currency, basePrice, unitType, minPax, maxPax, validFrom, validTo, validDays, blackoutDates, durationMinutes, timeOfDay, pickup and drop-off details, inclusions, exclusions, travellerRules, and cancellationRules. Prisma creates the RatePlan and its nested TravellerRule and CancellationRule rows in one logical operation.

Save inventory

POST /api/availability/bulk sends a list of slots. Each slot contains ratePlanId, slotDate, startTime, capacity, available, closed, priceOverride, and expectedVersion. The expectedVersion prevents an older browser tab from overwriting a newer edit. A successful response is followed by a reload so the displayed values come from PostgreSQL.

Field behavior and validation rules

Area	Field	Expected value	Validation or business rule
Onboarding	Legal Business Name	Registered vendor name	Stored against the current VendorProfile
Onboarding	GSTIN	Tax registration number	Optional text in this demo; document verification is separate
Onboarding	Payout Account	Masked account display	Never used as a raw bank

			credential
Documents	File	PDF, JPG, JPEG, or PNG	The demo records filename, status, and upload time in documentStatus
Activity	Star Rating	0 through 5, decimals allowed	Backend limits the value to the supported range
Activity	Latitude / Longitude	Decimal coordinates	Converted to numeric values before persistence
Rate Plan	Base Price and traveller prices	Zero or positive numbers	Stored as PostgreSQL Decimal values
Rate Plan	Min Pax / Max Pax	Positive integers	Max Pax cannot be lower than Min Pax
Rate Plan	Valid From / Valid To	Dates	Valid To must be on or after Valid From
Rate Plan	Blackout Dates	Comma-separated ISO dates	Excluded dates remain visible as rate-plan configuration
Rate Plan	Cancellation slabs	Percentage values	Each charge is stored as a structured rule, not free text
Availability	Available / Capacity	Non-negative integers	Capacity and available are bounded by business inventory logic
Availability	Price Override	Optional non-negative number	If blank, rate-plan base price is used
Availability	Closed	On or off	Closed slots remain stored but are not saleable
Promotion	Discount %	0.01 through 100	Start and end define the active time window

What changes in the database during the demo

Demo action	Primary table	Related tables or fields	How to prove it
Save onboarding	VendorProfile	legalBusinessName, GSTIN, category, payoutAccountMasked	Refresh Onboarding and open VendorProfile in pgAdmin
Upload a document	VendorProfile.documentStatus JSON	key, fileName, status, uploadedAt	Open the JSON column and find the document key
Save an activity	Activity	terms, highlights, thingsToCarry, importantInfo, lat, lon, starRating	Refresh the editor and inspect Activity
Add media	ActivityMedia	activityId, kind, url, rank	Open the ActivityMedia rows for the activity
Add rate plan	RatePlan	TravellerRule and CancellationRule	Match ratePlanCode to its child rows
Save inventory	AvailabilitySlot	capacity, available, closed, priceOverride, version	Match ratePlanId, date, and startTime
Schedule promotion	Promotion	activityId, discountPercent, startsAt, endsAt, active	Open Promotion and compare the displayed card
Confirm booking	Booking	status and version	Refresh Bookings and inspect status CONFIRMED

Error and recovery scenarios

Scenario	Expected result	How to recover
Wrong login password	Login remains on screen with an error	Use vendor@voya.demo and Demo@123
Backend unavailable	Screen shows a request error or cannot load	Start the backend on port 4007 and reload
Invalid rate-plan dates	Create action is rejected	Choose a Valid To date on or after Valid From
Max Pax below Min Pax	Client-side error appears before submission	Increase Max Pax or reduce Min Pax
Expired or missing JWT	API returns 401	Sign in again
Editing stale availability	API returns a conflict	Reload Availability and apply the change again
Voucher requested for pending booking	API refuses the request	Confirm the booking first
Publishing an already live activity	Status transition is rejected or unnecessary	Edit, submit, and publish only when a new review is required

Narrated demonstration explanation

Begin by saying: “This is a tenant-aware vendor console. I will create a business-to-booking flow. I will first authenticate, then update vendor readiness, maintain a sellable activity, attach a commercial rate plan, add inventory, accept a booking, and finally verify the rows in PostgreSQL.”

At Onboarding, explain that the vendor profile is the identity anchor. At Listings, explain that an Activity is the product master and does not itself define every price. At the Activity Editor, explain that one activity can have several rate plans, for example shared, private, child, premium, or vehicle-specific offers. At Availability, explain that the selected rate plan determines which inventory grid is being edited. At Bookings, explain that the booking points back to the activity and selected rate plan and moves through controlled states. Finish in pgAdmin to prove that the UI is connected to real stored data.

Role and ownership matrix

Capability	Vendor	Admin / Sub-admin	Reason
Read own profile and catalogue	Yes	Yes	Vendor operates its own data; platform roles can supervise
Create or edit own activity	Yes	Yes	Changes are tenant-scoped
Submit for review	Yes	No requirement in demo	Vendor requests operational approval
Publish in this demo	Yes	Yes	Enabled for demonstrating the complete lifecycle
Manage own inventory and promotions	Yes	Yes	Inventory belongs to the vendor’s rate plans
Confirm or cancel own bookings	Yes	No requirement in demo	Vendor fulfils reservations
Access another tenant’s rows	No	Platform policy dependent	Service filters enforce tenant ownership

Frontend component flow

The frontend is intentionally small and feature-oriented. App.tsx owns authentication, the selected menu page, the shared shell, the vendor profile summary, and sign out. Each page owns its loading, error, and data state.

ActivityEditor is embedded by Listings when a card or New Listing is selected. This means the browser can move from the catalogue list to the product editor without losing the parent listing data.

React file	Owns	Important local state	Server calls
App.tsx	Login, session, sidebar, current page	user, page, profile, checking	/auth/me, /vendor/profile, /auth/login
Onboarding.tsx	Business and document controls	profile, saved, error	GET/PATCH /vendor/profile, POST /vendor/documents/{key}
Listings.tsx	Search, filters, cards, editor entry	rows, search, filter, editing	GET /activities
ActivityEditor.tsx	Activity and rate-plan forms	form, media, ratePlans, saving, showRate	Activity, media, rate-plan, submit, publish endpoints
Availability.tsx	Plan selector, slot grid, promotions	slots, selectedRatePlanId, promos, promoForm	Availability and promotion endpoints
Bookings.tsx	Booking table and actions	rows, filter, error	GET bookings, confirm, cancel, voucher
Payouts.tsx	Settlement summary and table	rows, totals	GET payouts
Performance.tsx	Quality scorecard	summary, error	GET dashboard/summary

Navigation and data sequence

The following sequence is the recommended explanation while clicking through the portal:

From	User selects	To	Data relationship explained
Login	Sign in	Dashboard	JWT session allows protected requests
Dashboard	Onboarding	Vendor profile	Business identity and readiness are maintained before selling
Onboarding	Listings	Activity catalogue	The vendor profile belongs to the same tenant as the activities
Listings	Activity card	Activity Editor	The card id loads the complete activity and its related rate plans
Activity Editor	Add Rate Plan	Rate plan form	The rate plan receives the current activity id
Activity Editor	Availability menu	Availability grid	The grid is filtered by ratePlanId
Availability	Bookings menu	Booking queue	Bookings reference activity and optional rate plan
Bookings	Payouts	Settlement history	Payout rows are tenant-scoped financial summaries
Any screen	Performance	Quality scorecard	Metrics summarize operational behavior across the tenant

Backend module map

NestJS module	Controller prefix	Service responsibility	Prisma models
AuthModule	/auth	Login and current-user identity	User, Tenant
VendorModule	/vendor	Profile updates and document metadata	VendorProfile
ActivitiesModule	/activities	Catalogue CRUD, media, submission, publishing	Activity, ActivityMedia
RatePlansModule	/activities/{id}/rate-plans	Commercial plan creation and nested rules	RatePlan, TravellerRule, CancellationRule
AvailabilityModule	/availability and /promotions	Optimistic slot updates and promotions	AvailabilitySlot, Promotion
BookingsModule	/bookings	Booking list, confirmation, cancellation, voucher	Booking
PayoutsModule	/payouts	Tenant settlement history	Payout
DashboardModule	/dashboard	Summary metrics and recent bookings	Booking, Activity, VendorProfile

Protected endpoint contract

HTTP method	Endpoint	Role	Purpose	Success result
POST	/api/auth/login	Public	Authenticate demo user	201 with accessToken and user
GET	/api/auth/me	Authenticated	Restore session identity	200 user
GET	/api/vendor/profile	VENDOR	Load profile	200 VendorProfile
PATCH	/api/vendor/profile	VENDOR	Save business details	200 updated profile
POST	/api/vendor/documents/{key}	VENDOR	Register selected document	201 updated documentStatus
GET	/api/activities	VENDOR/ADMIN/SUB_ADMIN	List catalogue	200 activities with rate-plan summary
POST	/api/activities	VENDOR	Create draft activity	201 activity
PATCH	/api/activities/{id}	VENDOR/ADMIN/SUB_ADMIN	Update activity	200 activity
POST	/api/activities/{id}/media	VENDOR/ADMIN/SUB_ADMIN	Add media URL	201 media row
POST	/api/activities/{id}/rate-plans	VENDOR/ADMIN/SUB_ADMIN	Create commercial plan	201 rate plan and child rules
POST	/api/activities/{id}/submit	VENDOR	Move draft to review	201 activity with UNDER_REVIEW
POST	/api/activities/{id}/publish	VENDOR/ADMIN/SUB_ADMIN	Make activity live	201 activity with LIVE
GET	/api/availability	VENDOR/ADMIN/SUB_ADMIN	Load inventory	200 slots with rate-plan relation
POST	/api/availability/bulk	VENDOR/ADMIN/SUB_ADMIN	Save inventory grid	201 updated slots
POST	/api/promotions	VENDOR/ADMIN/SUB_ADMIN	Create promotion	201 promotion
PATCH	/api/promotions/{id}	VENDOR/ADMIN/SUB_ADMIN	Toggle active flag	200 promotion
GET	/api/bookings	VENDOR	Load booking queue	200 bookings
POST	/api/bookings/{id}/confirm	VENDOR	Confirm pending booking	201 booking
POST	/api/bookings/{id}/cancel	VENDOR	Cancel pending or confirmed booking	201 booking
GET	/api/bookings/{id}/voucher	VENDOR	Return voucher	200 voucherCode

			data	and booking
GET	/api/payouts	VENDOR	Load settlement rows	200 payouts

Database data dictionary

Table	Key columns	Relationship	Example meaning
Tenant	id, name, slug, kind	Parent of most vendor data	One vendor organization
User	id, tenantId, email, role, active	Belongs to Tenant	vendor@voya.demo
VendorProfile	tenantId, legalBusinessName, gstin, documentStatus	One-to-one with Tenant	Business verification record
Activity	id, tenantId, productName, status, location, content arrays	Belongs to Tenant	Sunrise Trek
ActivityMedia	activityId, kind, url, rank	Belongs to Activity	Cover, gallery, or video URL
RatePlan	id, activityId, ratePlanCode, price, valid dates	Belongs to Activity	Standard Rate Plan
TravellerRule	ratePlanId, type, age, count, price	Belongs to RatePlan	Adult, Child, Senior
CancellationRule	ratePlanId, day range, charge	Belongs to RatePlan	50% charge 1 to 5 days before
AvailabilitySlot	ratePlanId, slotDate, startTime, capacity, available, closed, version	Belongs to RatePlan	06:00 inventory on a date
Promotion	activityId, discount, window, active	Belongs to Activity	Low Occupancy Booster
Booking	tenantId, activityId, ratePlanId, status, amount	References Activity and RatePlan	Pending reservation
Payout	tenantId, amount, status, dueDate	Belongs to Tenant	Scheduled settlement

Implementation decisions and rationale

Decision	Implementation	Reason
Separate frontend and backend	activity-saas-frontend and activity-saas-backend	Each layer can be installed, built, and deployed independently
API prefix	/api	Keeps browser routes and API routes distinct
JWT authentication	Bearer token stored as voya_token	Allows page refresh without a server-side browser session
DTO validation	NestJS ValidationPipe with whitelist and transform	Rejects unsafe or incorrectly typed input
Tenant filtering	requireTenant in feature services	Prevents cross-vendor reads and writes
Nested rate-plan rules	Prisma nested create	Keeps traveller and cancellation rules attached to their plan
Optimistic inventory version	expectedVersion on AvailabilitySlot	Protects against lost updates from two browser tabs
Arrays for content	highlights, terms, inclusions, exclusions, thingsToCarry	Matches the structured catalogue model while accepting one-per-line input
JSON for document metadata	VendorProfile.documentStatus	Stores file name and verification

		metadata without exposing bank or document bytes
--	--	--

Setup and operational commands

From the project root, install each application independently. Start PostgreSQL first, then run Prisma migration and seed commands in the backend. Start the NestJS API on port 4007 and the Vite frontend on port 3007. The demo account is created by the seed script and the availability seed provides at least 10 rate plans with slots.

Purpose	Directory	Command
Install backend	activity-saas-backend	npm install
Generate Prisma client	activity-saas-backend	npm run prisma:generate
Apply migrations	activity-saas-backend	npm run prisma:migrate
Seed demo data	activity-saas-backend	npm run prisma:seed
Run backend	activity-saas-backend	npm run start:dev
Install frontend	activity-saas-frontend	npm install
Run frontend	activity-saas-frontend	npm run dev
Build backend	activity-saas-backend	npm run build
Build frontend	activity-saas-frontend	npm run build

QA acceptance checklist

ID	Test	Expected result
AUTH-01	Open the frontend without a token	Login screen appears
AUTH-02	Submit valid demo credentials	Dashboard opens and token is stored
AUTH-03	Submit an incorrect password	Error appears and no dashboard is shown
ONB-01	Change business name and save	Success message appears and value survives refresh
ONB-02	Select a document	Card shows verified status and filename
ACT-01	Create a listing with rating and coordinates	Listing appears with saved values
ACT-02	Add media URL	Media appears in the editor and persists after reload
ACT-03	Create a rate plan with Senior price	Plan and three traveller rules are returned
ACT-04	Enter invalid plan dates	Client or API rejects the request
ACT-05	Publish a draft	Listing status becomes LIVE
AVL-01	Select each seeded plan	At least 10 plans appear in the selector
AVL-02	Change capacity and available seats	Values remain after Save Inventory and refresh
AVL-03	Set price override and Closed	Both values persist on the slot
AVL-04	Create and stop a promotion	Promotion card changes between Live now and Stopped
BKG-01	Confirm pending booking	Status changes to CONFIRMED
BKG-02	Request voucher for confirmed booking	Voucher code is returned
BKG-03	Request voucher for pending booking	API rejects the request
DB-01	Inspect rows in pgAdmin after a browser save	Corresponding PostgreSQL row contains the same value

How the screens are linked

The left menu is available after login. Each menu item changes the main screen without changing the vendor account. Several screens are linked by IDs: an Activity owns Rate Plans; a Rate Plan owns Traveller Rules, Cancellation Rules, and Availability Slots; Bookings can refer to an Activity and Rate Plan.

Screen	Main purpose	What it links to
Dashboard	Summary of vendor activity	Bookings, Listings, Onboarding
Onboarding	Business verification and payout readiness	VendorProfile and documents
Listings	Search and open activities	Activity Editor and Rate Plans
Activity Editor	Create/edit product information	Media, Rate Plans, Availability
Availability	Set dates, slots, capacity and promotions	RatePlan and AvailabilitySlot
Bookings	Confirm or cancel reservations	Activity, RatePlan and Booking
Payouts	View settlement history	Payout
Performance	Review service quality	Dashboard and booking metrics

Login and dashboard

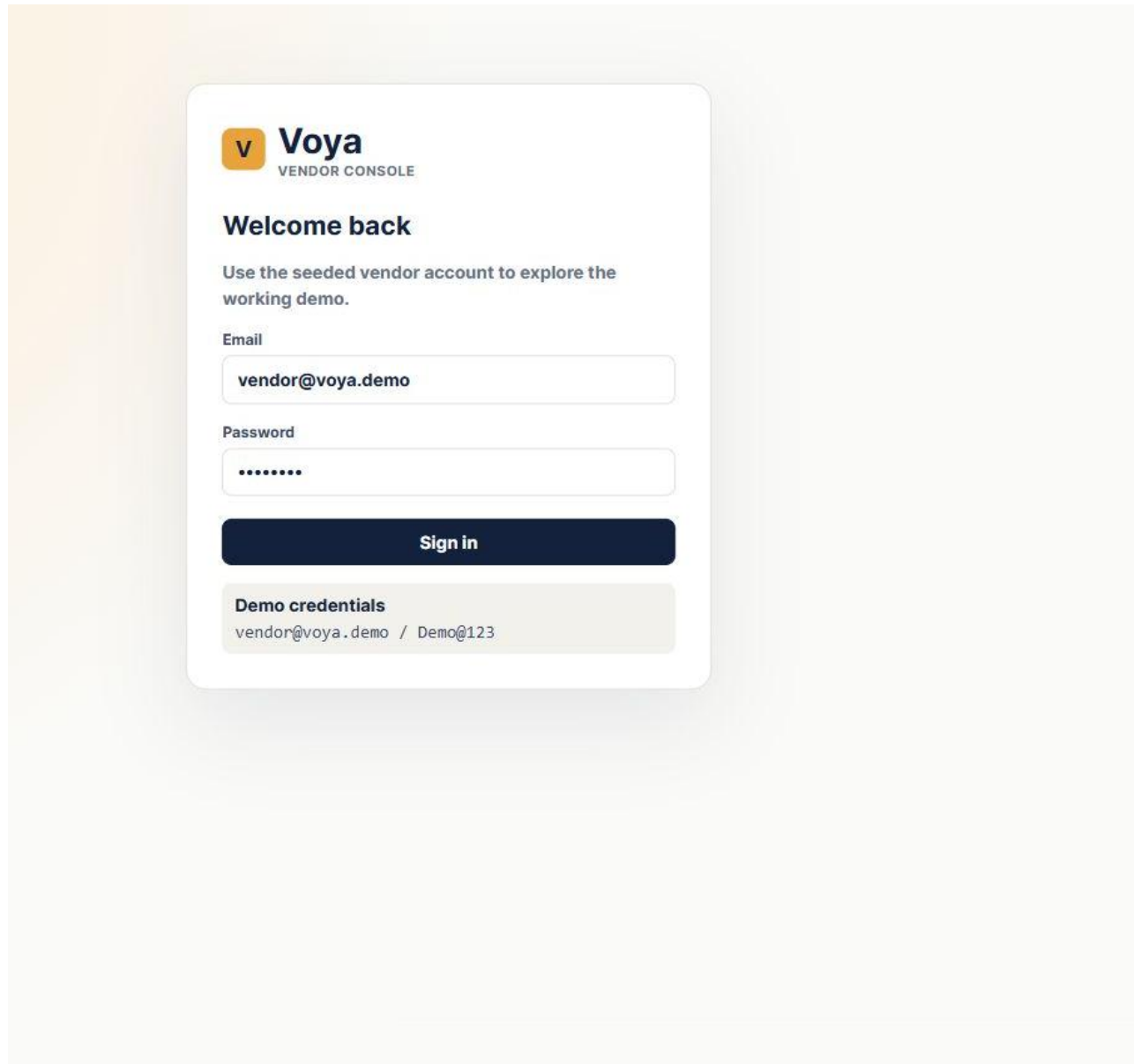


Figure 1. Exact login screen captured from the running Chrome application.

Enter the seeded demo email and password, then select Sign in. Authentication is performed by the NestJS API and the returned JWT is stored by the browser for authenticated requests; the React app does not fake a successful login.

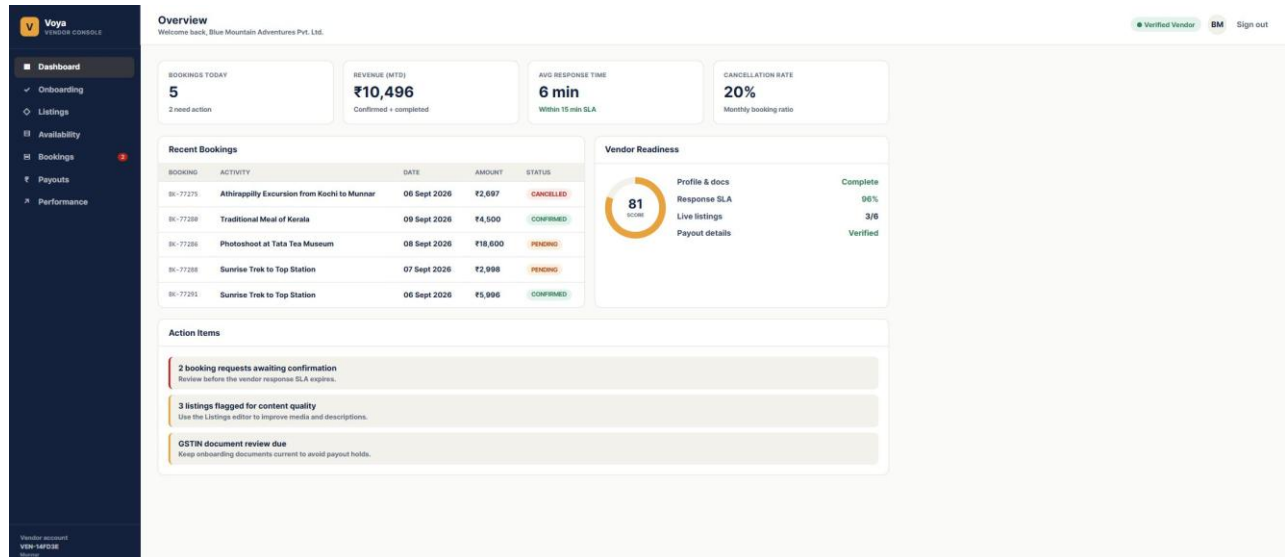


Figure 2. Exact dashboard screen captured from the running Chrome application. The dashboard is the starting point after a successful login.

The dashboard gives a quick operational summary. Bookings Today counts recent booking activity, Revenue MTD shows confirmed and completed revenue for the month, Pending Bookings needs vendor action, and Vendor Readiness shows how complete the account is.

Onboarding

Voya Vendor Console

Complete verification to unlock full catalogue access

Verified Vendor BM Sign out

Onboarding

1 Business Details
2 Documents
3 Bank & Payout
4 Catalogue Setup
5 Go Live Review

Business Details

Legal Business Name: Blue Mountain Adventures Pvt. Ltd.

Operating City: Munnar

Operating Region: Kerala

GSTIN: 32AACCB1234F1Z5

Category: Trekking, Nature & Wildlife

Payout Account: HDFC **** 4821

Save Business Details

Documents

- GSTIN Certificate: Uploaded - Verified (Replace)
- PAN Card: Uploaded - Verified (Replace)
- Cancelled Cheque / Bank Proof: Required / pending (Upload)
- Trade License / Activity Permit: Required / pending (Upload)

Vendor account: VEN-14PQ38

Figure 3. Exact onboarding screen captured from the running Chrome application.

Field	Meaning	Where it is stored
Legal Business Name	Registered business name	VendorProfile.legalBusinessName
Operating City / Region	Location used for vendor identity and operations	VendorProfile.operatingCity / operatingRegion
GSTIN	Tax registration identifier	VendorProfile.gstin
Category	Business category shown to the vendor	VendorProfile.category
Payout Account	Masked settlement account display	VendorProfile.payoutAccountMasked

Listings

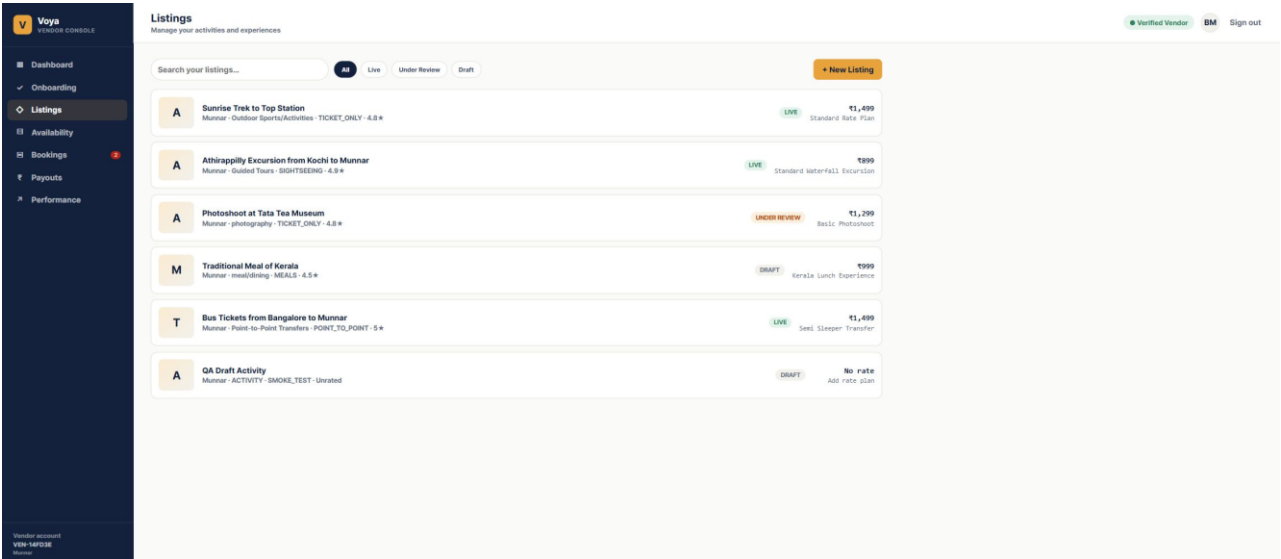


Figure 4. Exact listings screen captured from the running Chrome application.

Listings is the activity catalogue. Use the search box to find an activity, use status filters to narrow the list, and click a listing card to open the Activity Editor. “Live”, “Under Review”, and “Draft” are workflow states. A listing price is taken from its first rate plan when one exists.

Activity editor

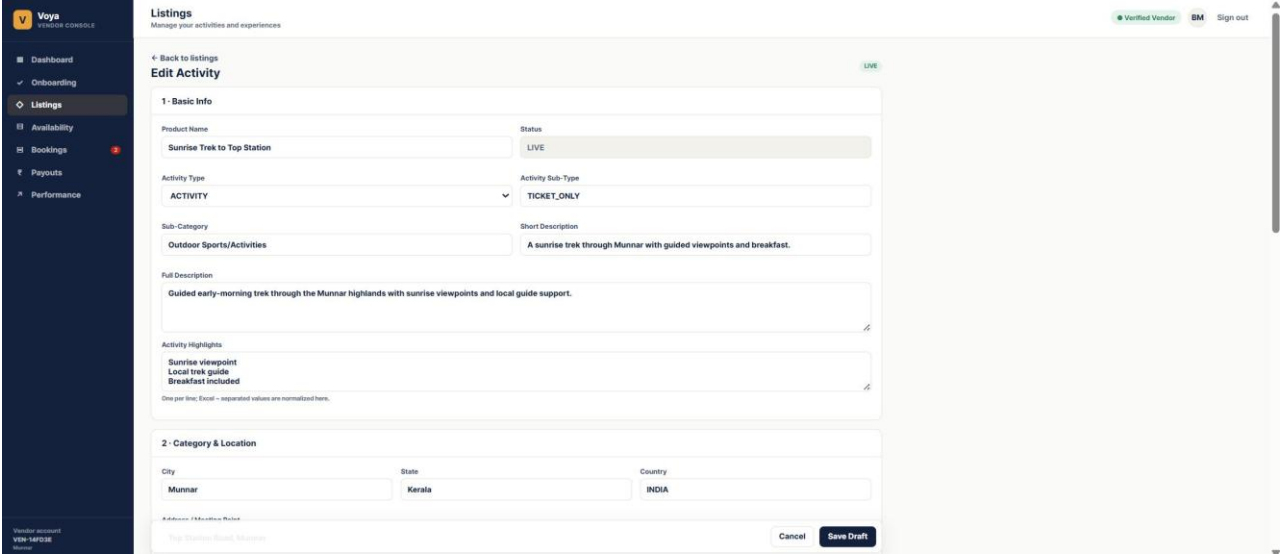


Figure 5. Exact Activity Editor screen captured from the running Chrome application.

Section	Important fields	What to enter
Basic Info	Product Name, Activity Type, Activity Sub-Type, Star Rating, Description	Customer-facing product identity and explanation
Category and Location	Sub-Category, City, State, Country, Address, Latitude, Longitude	Where the activity happens and how it is categorized
Media	Media Type, Media URL	Image or video URL for the product catalogue
Logistics	Things to Carry, Important Information	Operational instructions shown with the

		product
Rate Plan and Travellers	Rate plan code, currency, price, pax limits, traveller prices	Commercial offer that can be booked
Policies and Cancellation	Terms and cancellation slabs	Customer terms and charges based on days before service

Save Draft writes the activity to the database. Submit for Review changes a draft to Under Review. Publish Listing is available in the working demo and changes the listing to Live. Editing a live activity moves it back to review so production changes are checked.

Rate plans and traveller pricing

A rate plan is a bookable commercial version of an activity. One activity can have many rate plans, such as Standard, Private, Premium, Adult, Child, or different vehicle options. The activity is the product; the rate plan is the price and operating rules for that product.

Field	Meaning
Rateplan ID / Code	Unique code for the plan within its activity
Rateplan Name	Human-readable commercial name
Base Price	Default price stored as a database Decimal
Unit Type	Whether price is per person or per unit
Min Pax / Max Pax	Allowed booking group size
Valid From / Valid To	Date range when the plan can be used
Currency	Three-letter currency code used for the plan
Valid Days / Blackout Dates	Days the plan operates and dates that are excluded
Duration	Expected activity duration in minutes
Time of Day / Pickup Details	Operating time and meeting-point information
Pickup / Drop-off	Operational transport details
Inclusions / Exclusions	What is and is not included in the price
Traveller Rules	Adult, child, senior, and other age/price rules
Auto-Redeem Voucher	Whether the voucher is marked redeemed automatically
Cancellation Rules	Ordered slabs for cancellation charges

Availability and promotions

Voya
TRAVEL CONSOLE

Availability & Pricing
Manage slots, capacity and promotions

Verified Vendor BM Sign out

Sunrise Trek to Top Station — Standard Rate Plan

Schedule Promotion

Low Occupancy Booster — Sunrise Trek to Top Station
Time-based discount for low-occupancy inventory. The API persists and validates the promotion window.

Use now

DISCOUNT: 15% OFF
STARTS: 05 Sept 2026
ENDS: 07 Sept 2026
BOOKING CAP: 10

Active — click to stop early

Sunrise Trek to Top Station — Standard Rate Plan Slot Inventory

Save Inventory

SLOT	09 SEPT 2026	10 SEPT 2026	11 SEPT 2026	12 SEPT 2026	13 SEPT 2026	14 SEPT 2026	15 SEPT 2026	16 SEPT 2026
06:00	8 / 12 £1,499	8 / 12 £1,499	8 / 12 £1,499	8 / 12 £1,499	8 / 12 £1,499	0 / 12 £1,499	8 / 12 £1,499	8 / 12 £1,499
09:30	10 / 12 £1,499	10 / 12 £1,499	10 / 12 £1,499	10 / 12 £1,499	10 / 12 £1,499	2 / 12 £1,499	10 / 12 £1,499	10 / 12 £1,499

Standing Pricing Rules

Rule	Condition	Discount	Status
Weekend Surcharge	Sat-Sun, all slots	+15%	Active
Early Bird (> 7+ days)	All listings	-10%	Active

These illustrative standing rules are UI-only in this demo. Promotions and slot inventory are fully persisted; rule-engine configuration is intentionally left for the next approved scope.

Vendor account
VEN-140236
Sunrise

Figure 6. Exact Availability screen captured from the running Chrome application. The live demo contains at least 10 selectable rate plans with inventory.

Field	Meaning	Rule
Rate Plan Selector	Chooses which plan's slots are being viewed	Only rate plans with seeded inventory appear
Slot Date	Service date	Stored as a date without a time of day
Start Time	Daily operating slot time	Combined with date and plan to identify a slot
Available	Remaining places for that slot	Must not be greater than capacity
Capacity	Maximum places for that slot	Used to protect inventory limits
Price Override	Optional slot-specific price	Otherwise the rate plan base price is used
Closed	Temporarily closes a slot	Closed slots cannot be sold
Promotion	Time-boxed discount for an activity	Has start, end, discount, and booking cap

Save Inventory sends the visible slots to the API. The backend uses optimistic version checks so a stale browser cannot silently overwrite another update.

Bookings

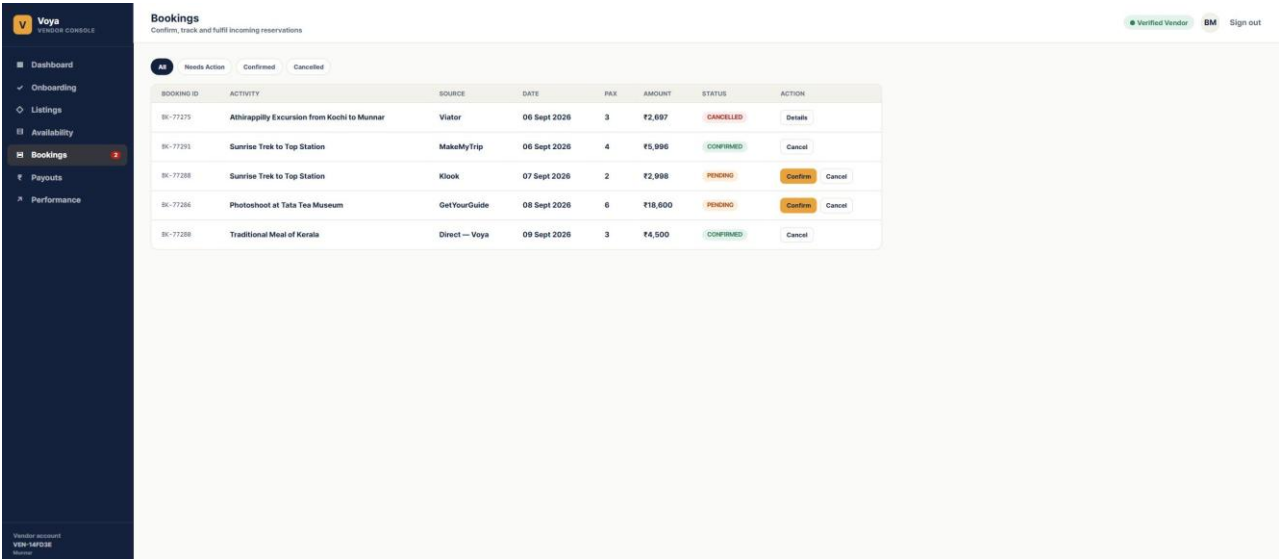


Figure 7. Exact Bookings screen captured from the running Chrome application.

Use All, Needs Action, Confirmed, and Cancelled filters to manage incoming reservations. A pending booking can be confirmed or cancelled. Confirmed and completed bookings expose Voucher. The API protects valid state transitions and keeps booking records isolated to the vendor tenant.

Payouts and performance

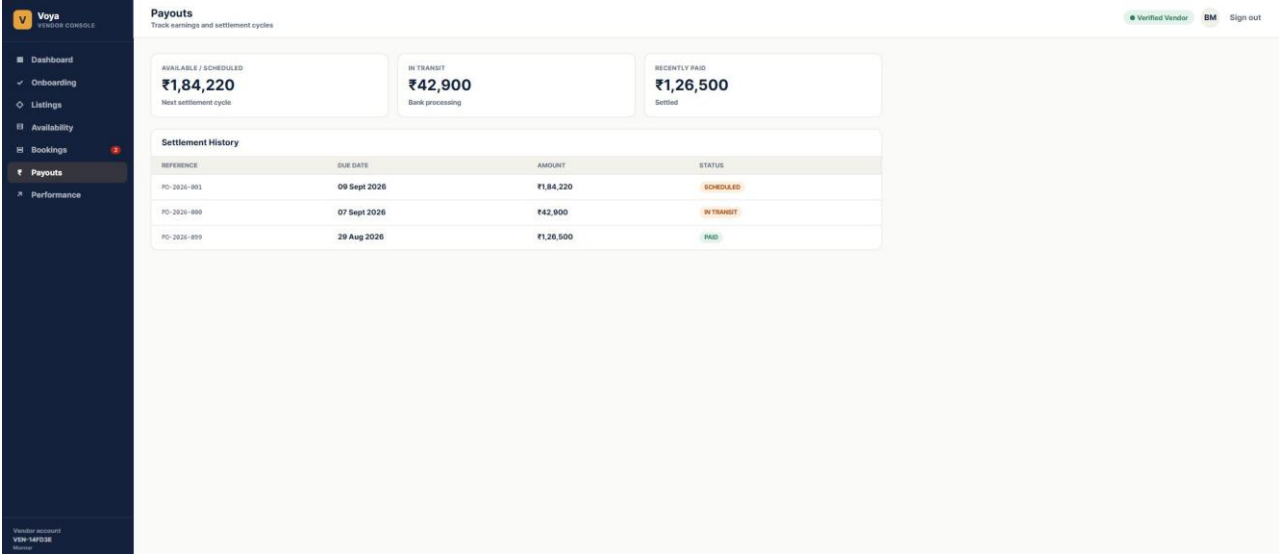


Figure 8. Exact Payouts screen captured from the running Chrome application.

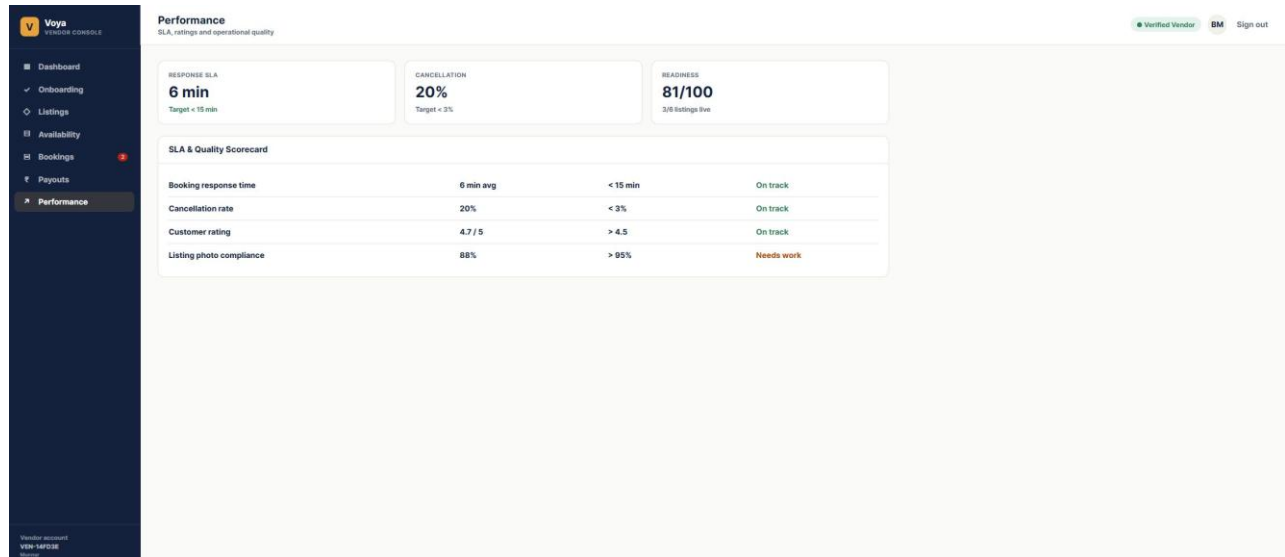


Figure 9. Exact Performance screen captured from the running Chrome application.

Payouts is read-only settlement history in this demo. Performance summarizes response time, cancellation rate, customer rating, readiness, and listing quality so the vendor knows what needs attention.

Recommended live demo script

This is a short logical demonstration for a stakeholder or tester. Keep the browser and pgAdmin open side by side when you want to prove that the UI change reached PostgreSQL.

Order	What to do	What to point out
1	Open http://localhost:3007 and sign in with <code>vendor@voya.demo</code> / <code>Demo@123</code>	The login is real and creates an authenticated session.
2	Open Onboarding and change the category, then save	The success message confirms the PATCH request; refresh to prove persistence.
3	Choose a pending document and select a file	The card changes to Uploaded and Verified with the filename.
4	Open Listings, select an activity, and review the editor sections	Basic information, location, media, logistics, rate plans, policies, and availability are linked by the activity ID.
5	Add or edit a rate plan	Explain currency, traveller prices, pax limits, valid dates, operating days, blackout dates, cancellation rules, and auto-redeem.
6	Select Publish Listing	The activity becomes LIVE in the demo and returns to the listing list.
7	Open Availability and choose one of the 10 seeded plans	Edit available seats, capacity, price override, and Closed, then select Save Inventory.
8	Schedule a promotion and toggle it	The time-boxed promotion is persisted and can be stopped or resumed.
9	Open Bookings and confirm a pending booking	The status changes to CONFIRMED; select Voucher to show the generated voucher code.
10	Return to Dashboard, Payouts, and Performance	These screens read summary and settlement data from the API and show the operational result.

How to verify the flow in pgAdmin

Connect to PostgreSQL 16, expand Databases, select activity_saas, then expand Schemas, public, and Tables. Right-click a table and choose View/Edit Data followed by All Rows. Useful tables for the demonstration are VendorProfile for onboarding and documents, Activity for listing fields and status, RatePlan for commercial rules, TravellerRule and CancellationRule for nested pricing rules, AvailabilitySlot for inventory, Promotion for discounts, Booking for confirmations and vouchers, and Payout for settlement history.

The strongest verification is to edit one value in the browser, save it, refresh the screen, and then open the corresponding table row in pgAdmin. The value should be identical because the browser does not use a mock-only form state for these persisted actions.

Database relationship in simple terms

The important chain is: Tenant → Vendor User and Vendor Profile. Tenant → Activities. Activity → Media, Rate Plans, Promotions, and Bookings. Rate Plan → Traveller Rules, Cancellation Rules, and Availability Slots. Booking can point back to the Activity and the selected Rate Plan.

When you use this screen	The main database row being changed
Save Business Details	VendorProfile
Save Draft / Edit Activity	Activity
Add Media	ActivityMedia
Create Rate Plan	RatePlan plus TravellerRule and CancellationRule
Save Inventory	AvailabilitySlot
Schedule Promotion	Promotion
Confirm or Cancel Booking	Booking

Troubleshooting

If the browser is blank, confirm the backend is running on port 4007 and the frontend on port 3007. If lists are empty, confirm the browser is logged in as vendor@voya.demo and that PostgreSQL database activity_saas is running. In pgAdmin, browse rows by expanding activity_saas → Schemas → public → Tables, then right-click a table and choose View/Edit Data → All Rows.